MANUFACTURING ANALYTICS / TECHNICAL CASE STUDY

Manufacturing Cycle Time Analytics Board

How I got four teams to agree on one cycle-time number, and what it unlocked across the site.

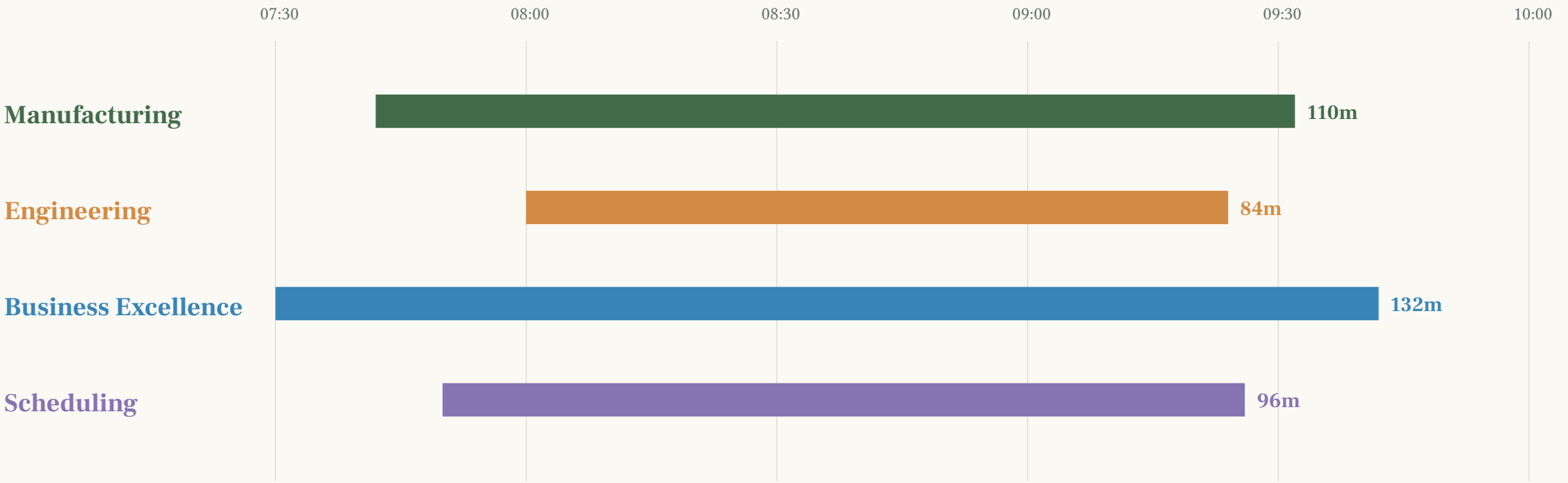
TYLER VINCENT / ANALYTICS ENGINEERING

Illustrative facility imagery

Inconsistent definitions steered investment decisions.

Growing demand and tight capacity: produce more with the same equipment and staff.

How long does filtration take? Four teams used different start and end points.



CURRENT ISSUE

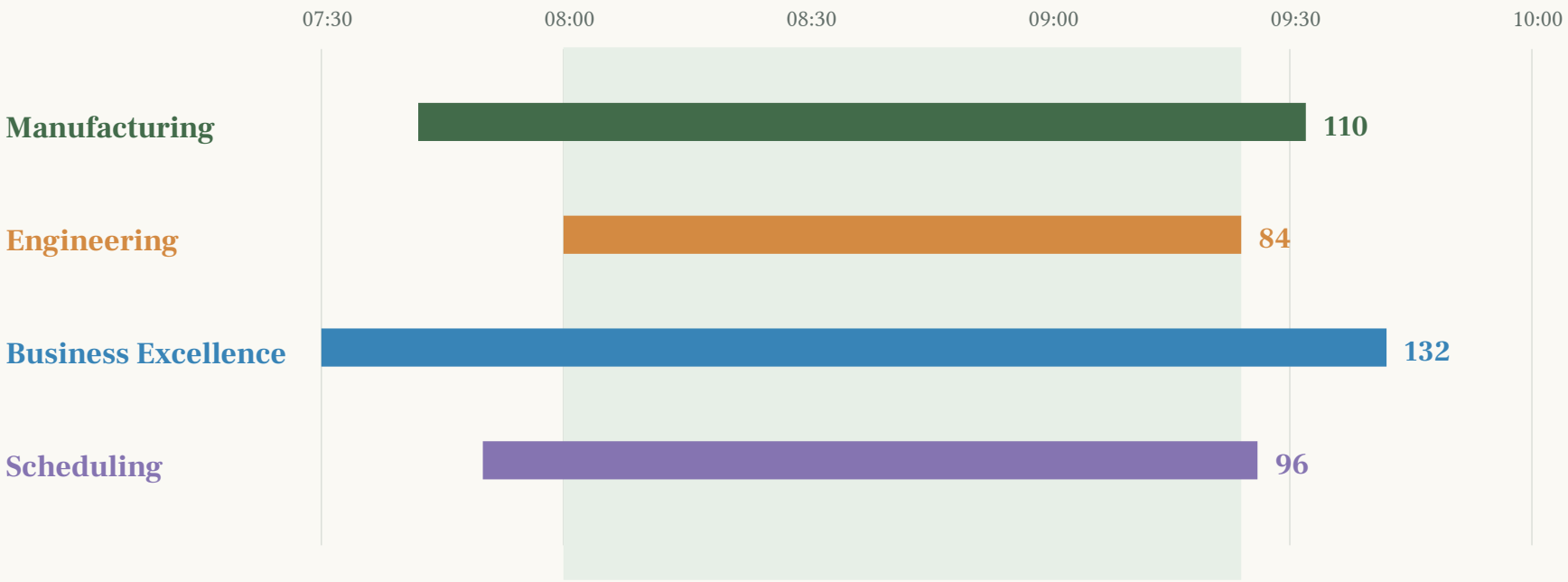
Spreadsheet choice decided which bottleneck got funded.

Illustrative intervals and durations; all values are synthetic.

One filtration step. Four numbers. One agreed definition.

A single step inside the filter station, shown with illustrative timestamps.

My proposal: agree the boundary, then certify one number.



CERTIFIED

84 min

START 08:00 RUNNING

END 09:24 COMPLETE

Owner: Engineering

Setup and waiting time remain separately defined.

WORKSHOP · Four functions · Weekly reviews · Process-owner sign-off

Synthetic example only: definitions, timing, roles, and workshop decisions are illustrative.

Machine signals become one process-step record.

In-cycle prompts count within the 84 min. Pre-cycle setup and handoff wait are separate.

SOURCE EVENTS

Timestamp	State	Event
07:58	IDLE	Ready
08:00	**RUNNING**	**Start**
08:10	IDLE	Prompt
08:20	RUNNING	Resume
09:24	**COMPLETE**	**End**

STEP FACT

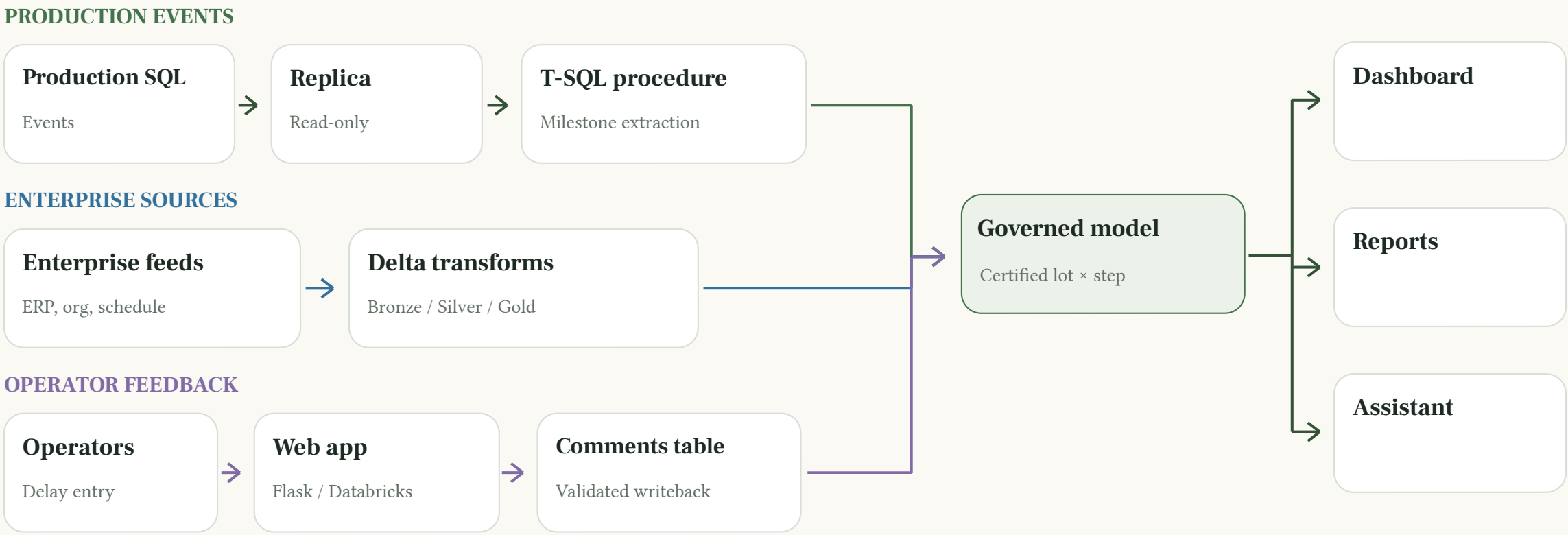
Lot	Lot 01
Step	Filter-station step
Start	**08:00**
End	**09:24**
Duration	**84 min**

VALUE STREAM

Step A | **Filter station** | Step C | Step D | Step E

Two source paths feed one governed model.

Production SQL, enterprise sources and operator comments converge in a governed reporting model.



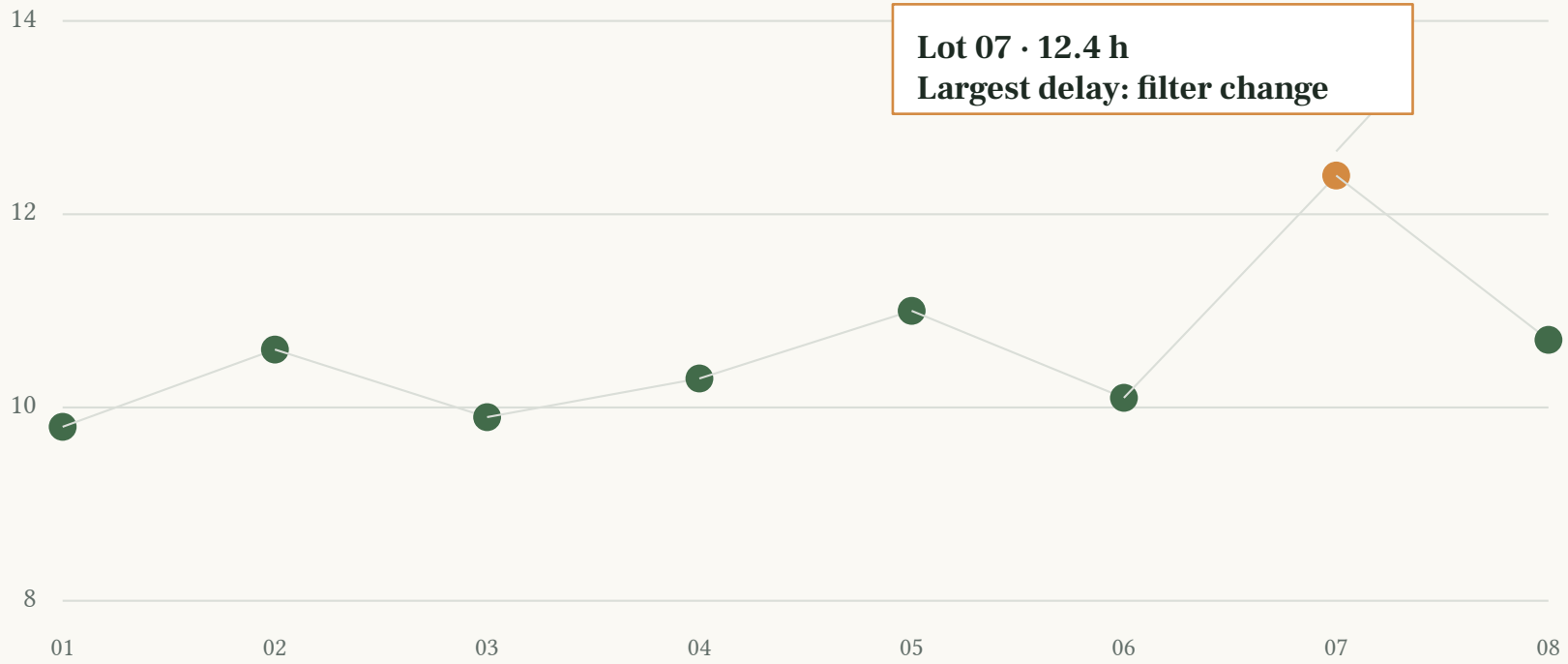
Data quality and access checks are applied before business consumption.

From a slow lot to a traceable delay.

Synthetic dashboard example: select Lot 07, inspect its deviation and review the delay comment.

Shift	All	Supervisor	All	Step	All	Period	Last 8 lots

LOT CYCLE TIME, ALL STEPS (HOURS)



SHIFT AVERAGE



Daily review: Lot 07 → identify the slow step → read the comment → assign a follow-up

The shared measures supported tangible results.

The board supplied a common baseline; improvement teams delivered the operational gains.

20% → 84%

Strategic projects meeting planned benefits

Share of reviewed projects delivering the intended result

- 54 min

Lot cycle time after filter-station fixes

10.8 → 9.9 hours per lot

~$1.5M

Annualized throughput value

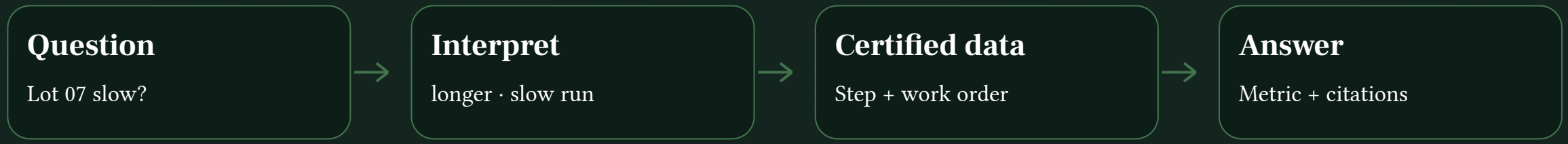
Reported value of implemented capacity improvements

HOW TEAMS USED IT

Manufacturing	Engineering	Business Excellence	Scheduling	IT
Staffing	Capital	Project selection	Targets and forecasts	Governed data

AI answers need a certified definition.

Question → synonyms → certified measure → evidence-backed answer



OPERATORS

Why did Lot 07 take longer at the filter station?

AI

ASSISTANT

Lot 07's filter-station step ran 22 minutes longer than its median. Operators logged a filter-change delay, and a related work order records a gasket replacement. Sources: PLC/control-system timestamps, operator-entered delay comments, and work orders.

Access-checked · cited sources · monitored answers · no equipment control

Illustrative example

What I'd keep, improve, and scale.

What I'd carry into the next analytics project.

WHAT I'D KEEP

Floor-first design

I'd involve technicians and operators early to validate process events and earn trust across the plant.

Also retain statistical checks and agent-assisted review to filter noisy signals.

WHAT I'D IMPROVE

Compute upstream

I'd move expensive transformations out of the BI front end, with monitored refreshes and clear failure handling.

Next: detect process drift, variation and emerging constraints proactively.

HOW I'D SCALE IT

Reusable governed data

I'd publish certified datasets that analysts and data scientists can reuse for trusted analysis and new applications.

Define alert severity, human escalation and useful/not useful feedback.

Next role: co-define metrics, certify trusted data, build tools, then add AI.

Technical appendix

Architecture · Semantic model · SQL · Tests · Versioning · Writeback

Detailed architecture

Two approved data routes converge in the certified analytics model.

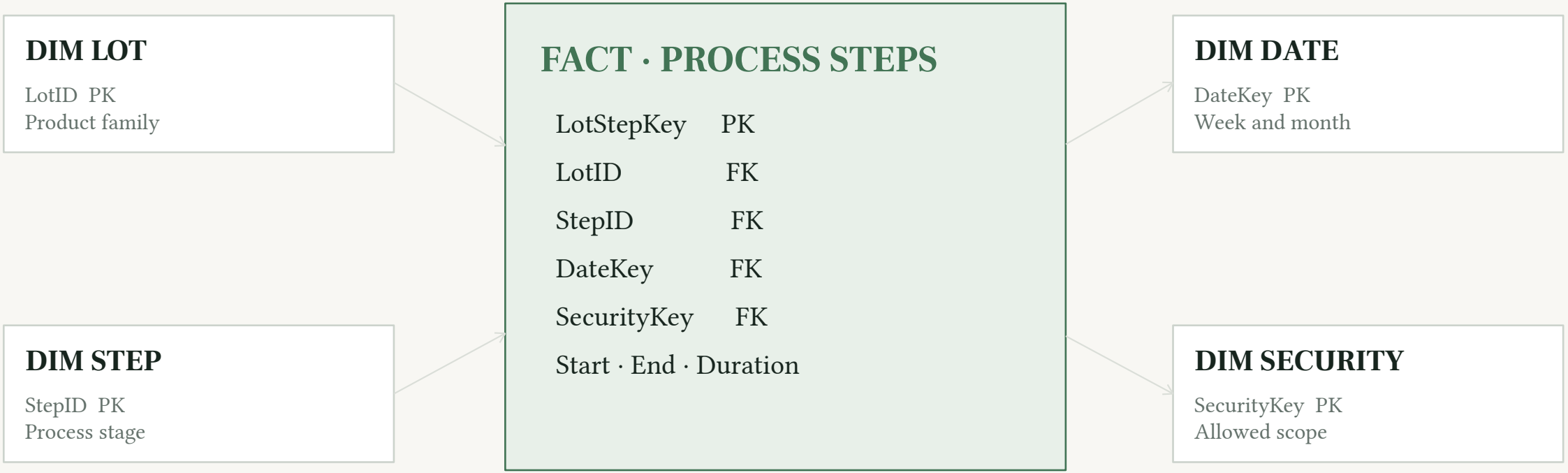


WRITEBACK: operator → Flask / Databricks app → Delta comments → joined in reporting model

Governance: reviewable SQL, Git changes, controlled service access, monitoring and source-owner approval.

Semantic model: one lot × one microstep.

Simplified star schema. Primary and foreign keys are explicit.



Conceptual model. Actual row-level security may rely on a separate user-scope bridge.

SQL: identify milestones at the right grain.

Illustrative T-SQL-style example, not production code.

```
WITH e AS (
  SELECT lot_id, step_id, attempt_id, event_time, state,
    ROW_NUMBER() OVER (
      PARTITION BY lot_id, step_id, attempt_id
      ORDER BY event_time DESC) AS last_row
  FROM normalized_events
), steps AS (
  SELECT lot_id, step_id, attempt_id,
    MIN(CASE WHEN state='RUNNING'
        THEN event_time END) AS start_time,
    MAX(CASE WHEN state='COMPLETE'
        THEN event_time END) AS end_time,
    MAX(CASE WHEN last_row=1
        THEN state END) AS last_state
  FROM e
  GROUP BY lot_id, step_id, attempt_id
)
SELECT steps.*,
  CASE WHEN start_time IS NULL
         OR end_time < start_time THEN 'INVALID'
       WHEN end_time IS NOT NULL THEN 'COMPLETE'
       WHEN last_state IN ('ABORTED','STOPPED') THEN 'ABORTED'
       ELSE 'IN_PROGRESS' END AS status
FROM steps;
```

IDENTITY

Normalize lot, step and attempt.

MATCH

Classify valid event boundaries.

TEST

Check duplicates, nulls, order and ranges.

Attempt IDs: hard stops start a new run.

Proposed SQL pattern: calculate previous state, then accumulate hard-stop restart flags.

```
WITH ordered AS (
  SELECT lot_id, step_id, event_time, event_id, state,
    LAG(state) OVER (
      PARTITION BY lot_id, step_id
      ORDER BY event_time, event_id
    ) AS prev_state
  FROM normalized_events
), flagged AS (
  SELECT *,
    CASE WHEN state='RUNNING'
       AND prev_state IN ('ABORTED','STOPPED')
       THEN 1 ELSE 0 END AS restart_flag
  FROM ordered
)
SELECT *,
  1 + SUM(restart_flag) OVER (
    PARTITION BY lot_id, step_id
    ORDER BY event_time, event_id
    ROWS UNBOUNDED PRECEDING
  ) AS attempt_id
FROM flagged;
```

IDLE → RUNNING

Resume within the same attempt.

STOPPED → RUNNING

Create the next attempt ID before aggregating start/end events.

KEY TEST

An idle prompt at 08:10 and resume at 08:20 must not split the 84-minute run.

Illustrative rule; verify source event semantics and stable event ordering before production use.

Event edge cases and tests.

Test cases I would add to harden the event rules.

RERUN

SOURCE EVENTS

10:00 start → 10:08 abort
10:20 restart → 11:40 end

EXPECTED RESULT

Completed attempt: 10:20–11:40

Test: do not join attempts

ABORT

SOURCE EVENTS

10:00 start → 10:30 abort
No completion received

EXPECTED RESULT

Null duration; aborted state

Test: require valid end

LATE EVENT

SOURCE EVENTS

11:30 complete arrives after reporting refresh

EXPECTED RESULT

Revisit affected lot/step

Test: overlap ✦ deduplicate

Proposed attempt rule: RUNNING after a hard stop begins a new attempt; RUNNING after IDLE remains the same attempt.

Validation: test code against operational reality.

A metric is publishable after source, SQL, and floor-level checks.

SOURCE	Counts and sampled timestamps	Event coverage; matched source rows
IDENTITY	Unique lot × step × attempt	No duplicate or mixed-lot facts
TIME	Start/end ordering and range	No negative or implausible durations
DRIFT	New state patterns and missing end	Alert / quarantine; trend exceptions
PEOPLE	Review on the floor	Operators and engineers confirm boundaries

Metric registry: business rule, owner, version.

An agreed definition becomes a reusable, documented contract.

METRIC	RULE	GRAIN	OWNER	EFFECTIVE	VERSION
Step duration	Complete minus valid start	Lot × microstep	Engineering	Oct 2026	v2
Handoff wait	Next start minus prior end	Lot × handoff	Manufacturing	Oct 2026	v1

Propose → Workshop → Validate → **Certify** → Publish → Review

Definition versioning and historical restatement.

A revised cycle-time boundary needs a deliberate consumer cutover.

v1 / ORIGINAL

Starts at equipment availability
Includes pre-cycle waiting

v2 / CERTIFIED

Starts at program running
Wait is a separate metric

DOWNSTREAM

Dual-run for comparison
Reconcile trends and alert owners

Choose whether to restate past records, preserve the v1 series, or publish both during transition.

Operator writeback: separate comments from raw events.

A controlled app captures context and joins it to measured durations.

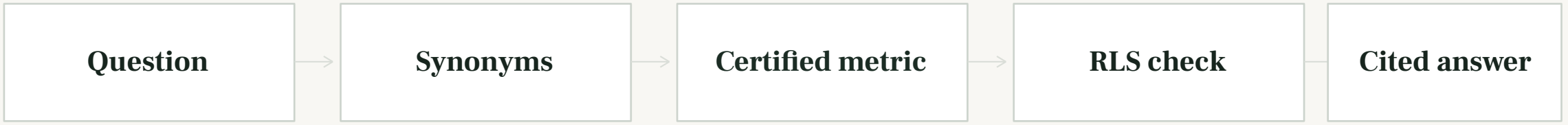


Allowed categories · User attribution · Timestamps · Access controls · Revision history

Only relevant delay signals and operator comments belong in the delays model. Other source systems feed the reporting model separately.

Assistant: certified answers and citations.

A permission-aware answer includes the metric version and citation.



BOUNDED SCOPE · ROLE-BASED RETRIEVAL · SOURCE CITATION · ANSWER MONITORING

Answers explain approved operational data. They do not operate equipment or independently change process instructions.

Copilot Studio: FAQ conversation topics · Teams access · session volume and drop-off monitoring

Medallion to dbt: transformations and quality tests.

Model layers and quality gates matter more than the specific tool.

DATA LAYER	MODEL OUTPUT	DBT ANALOG	TESTS
BRONZE	Raw, copied source events	source / staging	Source counts, not null
SILVER	Normalized events and identities	intermediate	Unique key, valid states
GOLD	Lot × microstep fact	mart	Unique, range, time-order
SEMANTIC	Certified cycle-time metric	metric / exposure	Reconciliation, permissions

Engineering improvement roadmap.

A plan for reliability, reuse and enterprise scale.

DELIVERED

Central SQL transformation
Shared metric review
Operational dashboard

NEXT RELEASE

Persist costly outputs
Exception + regression testing
Late-event policy

ENTERPRISE SCALE

Approved ingestion path
Monitoring and contracts
Reusable certified datasets

Constraints on the interim route.

Permissions, refresh expectations and source ownership shaped implementation.

Separated environments	Manufacturing systems and enterprise lake crossed security boundaries.
Governed connectivity	New lake feeds needed approved source integration and monitoring.
Timely operations	Manufacturing needed shift-level updates, not static CSV exports.
Maintainable changes	Database procedures were reviewable, testable and versionable in Git.
Long-term fit	Build now on approved routes, then migrate with validated consumers.